

Tensor Operations

A Problem Book

From First Principles to Attention Mechanisms

158 Problems in 13 Chapters

Basic Creation and Indexing – Broadcasting and Einsum –
Autograd – Applied Deep Learning Tensor Design

Answers Only, No Worked Solutions

Prepared for independent practice

July 13, 2026

Contents

How to Use This Book

This is a problem book, not a tutorial. Each chapter assumes you already know the relevant tensor operation and gives you volume and difficulty instead of explanation. Problems move from mechanical warm-ups to conceptual and applied questions that mirror how tensors actually get used in deep learning code.

Rules of the book:

- Work every problem on paper or in a notebook before checking the Answer Key at the back.
- Tensor values are written in NumPy/PyTorch-style bracket notation; assume `dim` in PyTorch corresponds to `axis` in NumPy.
- Where a problem is conceptual, the expected answer is a short, precise explanation — one or two sentences, not an essay.
- Do not skip the applied chapters at the end. Mechanical fluency with `reshape`/`broadcast`/`einsum` only matters once you can wire it into attention layers, losses, and convolutions.

On tools. There is no single "LeetCode for tensors" with the same scale and grading infrastructure, but the closest equivalents worth using alongside this book are:

- **Tensor Puzzles** (Sasha Rush) — a small, sharp set of NumPy/PyTorch broadcasting and indexing puzzles solved with one line each.
- **GPU Puzzles / Autodiff Puzzles** (Sasha Rush) — same format, aimed at kernel-level and gradient-level thinking once tensor mechanics are second nature.
- **NumPy 100 Exercises** (rougier/numpy-100 on GitHub) — broader NumPy drilling, useful for indexing/broadcasting reps.
- **Kaggle Learn / fast.ai exercises** — for tensor manipulation embedded in real model-building rather than isolated puzzles.
- Re-implementing pieces of a small transformer (attention, layernorm, cross-entropy) from raw tensor ops, with no library helpers, is the single best "final exam" once this book is finished.

Chapter 1

Tensor Creation & Attributes

- 1.1 Create a 1-D tensor containing the integers 0 through 9. What is its shape and number of dimensions?
- 1.2 A tensor T has shape (4,3,2). How many total elements does it contain, and what is its number of dimensions (rank)?
- 1.3 You create a tensor with `torch.zeros(5,5)`. State its `dtype` (default) and its shape.
- 1.4 What is the shape of the tensor produced by `torch.eye(4)`? Describe its values.
- 1.5 A tensor is created with `torch.arange(2,20,3)`. List all its elements and give its shape.
- 1.6 `torch.linspace(0,1,5)` generates 5 evenly spaced values from 0 to 1 inclusive. List them.
- 1.7 What is the difference in output shape between `torch.rand(3)` and `torch.rand(3,1)`? State both shapes.
- 1.8 Given a tensor `x` of shape (2,3,4), what does `x.numel()` return?

- 1.9** A tensor `x` has `dtype=torch.float32`. After `x = x.to(torch.int64)`, what happens to the value 3.9?
- 1.10** What tensor does `torch.full((2,3), 7)` create? Give shape and values.
- 1.11** Explain the difference between `torch.tensor(data)` and `torch.as_tensor(data)` when `data` is already a NumPy array.
- 1.12** What device and `requires_grad` value does a freshly created `torch.ones(3,3)` have by default?
- 1.13** Create a tensor of shape `(3,3)` filled with values drawn from a standard normal distribution using the function name only (no output needed).
- 1.14** A tensor `x` has shape `(1,)`. Is it a scalar or a 1-D tensor? What is `x.item()` used for?
- 1.15** Given `x = torch.tensor([1,2,3])` and `y = torch.tensor([1.0,2.0])`, what are `x.dtype` and `y.dtype`?

Chapter 2

Indexing & Slicing

Setup for this chapter's problems:

```
A = np.arange(1,25).reshape(4,6)
```

- 2.1** Given A of shape $(4, 6)$ with values $1 \dots 24$ row-major, what is $A[0]$?
- 2.2** What is $A[:, 0]$?
- 2.3** What is $A[1:3, 2:5]$?
- 2.4** What is $A[-1, -1]$?
- 2.5** What is $A[:, :2, ::2]$?
- 2.6** What is $A[:, ::-1]$?
- 2.7** What does $A[1]$ return in terms of shape – a row vector or a $(1, 6)$ matrix?
- 2.8** What is the shape of $A[None, :, :]$?
- 2.9** Given boolean mask `mask = A > 12`, what does $A[\text{mask}]$ return (shape and description)?

- 2.10** What is `A[[0,2], [1,3]]` (advanced/fancy indexing selecting two coordinate pairs)?
- 2.11** What is the result and shape of `A[torch.tensor([0,0,3])]` (row gather with repeats)?
- 2.12** Explain why `A[1:3]` returns a *view* in PyTorch (shares memory) while `A[mask]` with a boolean mask returns a *copy*.
- 2.13** Given `A[..., 0]` where `A` has shape $(4, 6)$, what does the ellipsis `...` do here and what is the output shape?
- 2.14** What happens if you assign `A[0, :] = 0`? Describe the resulting first row and whether `A` is modified in place.

Chapter 3

Reshaping, Views & Permutations

Setup for this chapter's problems:

```
B = np.arange(24).reshape(2,3,4)
```

- 3.1** *B* has shape (2, 3, 4). What is the shape after `B.reshape(4,6)`?
- 3.2** What is the shape after `B.reshape(-1)`?
- 3.3** What is the shape after `B.reshape(2,-1)`?
- 3.4** What is the shape of `B.permute(2,0,1)`?
- 3.5** What is the shape of `B.transpose(0,1)` (swap axes 0 and 1)?
- 3.6** Given *x* of shape (1, 3, 1, 5), what is the shape after `x.squeeze()`?
- 3.7** Given *x* of shape (3, 5), what is the shape after `x.unsqueeze(1)`?
- 3.8** Why does calling `.view()` sometimes raise a runtime error while `.reshape()` does not on the same tensor?

- 3.9** After `y = B.permute(1,0,2)`, is `y` contiguous in memory? What must you call before you can safely `.view()` it?
- 3.10** What is the shape of `B.flatten(start_dim=1)`?
- 3.11** Given `x` of shape `(2, 3, 4, 5)`, what is the shape after `x.transpose(1,3)`?
- 3.12** What is the shape after `torch.movedim(x, 0, -1)` for `x` of shape `(2, 3, 4)`?
- 3.13** Given `x` of shape `(6, 4)` that you want reshaped to `(2, 3, 4)`, verify the reshape is valid by comparing total element counts. Is it valid?
- 3.14** What is the shape of `B.repeat(2,1,1)` for `B` of shape `(2, 3, 4)`? What is different about `B.expand(4,3,4)` started from a `(1, 3, 4)` tensor versus `repeat`?

Chapter 4

Broadcasting

- 4.1 Can a tensor of shape $(3, 4)$ be broadcast with one of shape $(4,)$? If so, what is the result shape?
- 4.2 Can a tensor of shape $(3, 1)$ be broadcast with one of shape $(1, 4)$? What is the result shape?
- 4.3 Can shapes $(3, 4)$ and $(3,)$ be broadcast together? Justify your answer using the broadcasting alignment rule.
- 4.4 What is the result shape of adding a tensor of shape $(5, 1, 3)$ and one of shape $(1, 4, 3)$?
- 4.5 What is the result shape of adding a tensor of shape $(8, 1, 6, 1)$ and one of shape $(7, 1, 5)$?
- 4.6 Given x of shape (N, D) and b of shape $(D,)$, what does $x + b$ compute conceptually, and what is a common name for this pattern in neural networks?
- 4.7 Explain why `torch.rand(3,4) * torch.rand(4)` works but `torch.rand(3,4) * torch.rand(3)` raises an error.
- 4.8 Two tensors have shapes $(1, 3, 1)$ and $(4, 1, 5)$. What is the broadcast result shape?

- 4.9** Given an image batch of shape $(N, 3, H, W)$ and per-channel mean of shape $(3, 1, 1)$, explain how normalization (`images - mean`) broadcasts.
- 4.10** Why can broadcasting silently produce incorrect results instead of an error when shapes have unintended size-1 dimensions? Give a concrete failure example.

Chapter 5

Elementwise & Arithmetic Operations

Setup for this chapter's problems:

```
x = np.array([1.0,-2.0,3.0,-4.0])
```

```
y = np.array([2.0,2.0,2.0,2.0])
```

5.1 Given $x = [1, -2, 3, -4]$, what is $x.abs()$?

5.2 What is x^2 (elementwise square)?

5.3 What is $\text{ReLU}(x) = \max(x, 0)$ elementwise?

5.4 What is `torch.sign(x)`?

5.5 What is $x \odot y$ (elementwise product) for $x = [1, -2, 3, -4]$,
 $y = [2, 2, 2, 2]$?

5.6 What is x/y elementwise?

5.7 What is `torch.clamp(x, min=-1, max=2)`?

5.8 What is the sigmoid $\sigma(x) = 1/(1 + e^{-x})$ applied element-

wise to $x = [1, -2, 3, -4]$? Round to 4 decimals.

- 5.9** What is $\tanh(x)$ for $x = [1, -2, 3, -4]$?
- 5.10** What is `torch.exp(x)` for $x = [1, -2, 3, -4]$?
- 5.11** Explain the difference between `x.add(1)` and `x.add_(1)` in PyTorch.
- 5.12** What is the GELU-style approximate output sign pattern of `torch.where(x > 0, x, 0.01*x)` (Leaky ReLU with slope 0.01) for $x = [1, -2, 3, -4]$?
- 5.13** What is $x\%3$ (elementwise modulo) for integer tensor $[7, 8, 9, 10]$?
- 5.14** What is `torch.pow(x, 3)` for $x = [1, -2, 3, -4]$?
- 5.15** What is the result and dtype of `torch.tensor([1,2,3]) / torch.tensor([2,2,2])` in PyTorch (true division vs integer division)?

Chapter 6

Reduction Operations

Setup for this chapter's problems:

```
M = np.array([[1.,2.,3.],[4.,5.,6.]])
```

6.1 Given $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$, what is `M.sum()`?

6.2 What is `M.sum(dim=0)`?

6.3 What is `M.sum(dim=1)`?

6.4 What is the shape of `M.sum(dim=1, keepdim=True)`?

6.5 What is `M.mean()`?

6.6 What is `M.max(dim=1)` – give both the values and the indices (`argmax`) along dim 1?

6.7 What is `M.argmax()` on the flattened tensor (single index into the flattened array)?

6.8 What is `M.var(dim=0, unbiased=False)` (population variance per column)?

6.9 What is `M.std(dim=0, unbiased=False)`?

- 6.10** What is `M.prod(dim=1)` (product along dim 1)?
- 6.11** What is `M.cumsum(dim=1)`?
- 6.12** Explain the difference in output shape between `M.sum(dim=1)` and `M.sum(dim=1, keepdim=True)`.
- 6.13** What is `torch.logsumexp(M, dim=1)`, and why is it numerically preferable to computing `torch.log(torch.sum(torch.exp(M), dim=1))` directly?
- 6.14** What is the result of `torch.all(M > 0)` and `torch.any(M > 5)`?

Chapter 7

Matrix Multiplication & Linear Algebra

Setup for this chapter's problems:

```
A = np.array([[1.,2.],[3.,4.]])
```

```
B = np.array([[5.,6.],[7.,8.]])
```

```
v = np.array([1.,1.])
```

7.1 Given $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, what is $A @ B$ (matrix product)?

7.2 What is $A \odot B$ (elementwise product, NOT `matmul`)?

7.3 What is A^T (transpose)?

7.4 What is $A @ v$ for $v = [1, 1]$?

7.5 What is $\det(A)$?

7.6 What is A^{-1} ?

-
- 7.7** What is $\text{trace}(A)$ (sum of diagonal)?
- 7.8** Given a batch of matrices X of shape $(16, 4, 5)$ and Y of shape $(16, 5, 3)$, what is the shape of `torch.bmm(X, Y)`?
- 7.9** Given X of shape $(4, 5)$ and Y of shape $(5, 3)$, what is the shape of $X @ Y$?
- 7.10** Given X of shape $(2, 4, 5)$ and a single matrix Y of shape $(5, 3)$, what is the shape of $X @ Y$, and what rule allows this without `bmm`?
- 7.11** What is the rank (via SVD or otherwise, conceptually) of a 3×3 matrix whose second row is a scalar multiple of its first row?
- 7.12** For $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, what are its eigenvalues (to 4 decimals)?
- 7.13** What is the Frobenius norm of $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$?
- 7.14** Explain why `torch.mm` only works on 2-D tensors while `torch.matmul` works on batched, higher-dimensional tensors too.

Chapter 8

Concatenation, Stacking & Splitting

Setup for this chapter's problems:

```
a = np.array([[1,2],[3,4]])
```

```
b = np.array([[5,6],[7,8]])
```

- 8.1** Given a, b each of shape $(2, 2)$, what is the shape of `torch.cat([a,b], dim=0)`?
- 8.2** What is the shape of `torch.cat([a,b], dim=1)`?
- 8.3** What is the shape of `torch.stack([a,b], dim=0)`? How does `stack` differ conceptually from `cat`?
- 8.4** Given a tensor of shape $(2, 2)$, what is `torch.cat([a,b], dim=0)` elementwise (the actual values)?
- 8.5** What is `torch.stack([a,b], dim=1)` in values, and what is its shape?
- 8.6** Given a tensor T of shape $(6, 4)$, what are the shapes of the outputs of `torch.split(T, 2, dim=0)`?

- 8.7** Given T of shape $(6, 4)$, what are the shapes of the outputs of `torch.chunk(T, 4, dim=0)` (splitting into 4 chunks along a dim of size 6)?
- 8.8** Why must all input tensors to `torch.cat` match exactly in every dimension except the concatenation dimension?
- 8.9** Given $\mathbf{x}$ of shape $(10, 3, 32, 32)$ representing a batch of images, how would you split it into two batches of 5 images each along the batch dimension, and what are the resulting shapes?

Chapter 9

Advanced Indexing: Gather, Scatter, Masking

Setup for this chapter's problems:

```
M = np.array([[10,20,30],[40,50,60],[70,80,90]])
```

```
idx = np.array([[0],[2],[1]])
```

9.1 Given $M = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$ and $\text{idx} = \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}$,

what does `torch.gather(M, 1, idx)` return (one element picked per row using `idx` as the column index)?

9.2 Explain conceptually what `torch.gather(input, dim, index)` does in one sentence.

9.3 Given `scores` of shape (N, C) (batch of class logits) and `labels` of shape $(N,)$, how would you use `gather` to extract the logit of the correct class for every sample?

9.4 What does `torch.scatter_(dim, index, src)` do, in contrast to `gather`?

- 9.5** How would you build a one-hot encoding of shape (N, C) from integer `labels` of shape $(N,)$ using `scatter`?
- 9.6** Given a boolean mask `mask` of shape $(3, 3)$ marking entries greater than 50 in M , what does `torch.where(mask, M, torch.zeros_like(M))` produce?
- 9.7** What is the fundamental difference between `torch.masked_select(M, mask)` and `torch.where(mask, M, other)`?
- 9.8** Given `index = torch.tensor([2, 0, 1])`, what does `torch.index_select(0, index)` return (rows of M reordered by index)?
- 9.9** Why is `gather` commonly used instead of a Python for-loop when extracting per-sample values (e.g. correct-class logits) in a training loop?

Chapter 10

Einsum & Tensor Contractions

Setup for this chapter's problems:

```
A = np.array([[1.,2.],[3.,4.]])
```

```
B = np.array([[5.,6.],[7.,8.]])
```

```
v = np.array([1.,2.])
```

```
w = np.array([3.,4.])
```

10.1 What does `torch.einsum('ij,jk->ik', A, B)` compute, in ordinary matrix notation?

10.2 What is the numeric result of `einsum('ij,jk->ik', A, B)` for $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$?

10.3 What does `torch.einsum('ii->', A)` compute?

10.4 What is the value of `einsum('ii->', A)`?

-
- 10.5** What does `torch.einsum('ij->ji', A)` compute?
- 10.6** What does `torch.einsum('i,i->', v, w)` compute, and what is its value for $v = [1, 2]$, $w = [3, 4]$?
- 10.7** What does `torch.einsum('i,j->ij', v, w)` compute, in ordinary terms?
- 10.8** What is the value of `einsum('i,j->ij', v, w)` for $v = [1, 2]$, $w = [3, 4]$?
- 10.9** Given `Q` of shape (B, H, N, D) and `K` of shape (B, H, N, D) (batch, heads, sequence length, head dim), what einsum expression computes the raw attention scores of shape (B, H, N, N) ?
- 10.10** Given attention weights `attn` of shape (B, H, N, N) and values `V` of shape (B, H, N, D) , what einsum expression produces the attention output of shape (B, H, N, D) ?
- 10.11** What does `torch.einsum('bij,bjk->bik', X, Y)` compute for batched `X` of shape $(B, 4, 5)$ and `Y` of shape $(B, 5, 3)$?
- 10.12** Why is `einsum` often preferred over chaining multiple `permute`, `unsqueeze`, and `matmul` calls when implementing custom tensor contractions?

Chapter 11

Norms, Distances & Similarity

Setup for this chapter's problems:

```
u = np.array([3.,4.])
```

```
v = np.array([0.,5.])
```

11.1 What is the L2 norm ($\|u\|_2$) of $u = [3, 4]$?

11.2 What is the L1 norm ($\|u\|_1$) of $u = [3, 4]$?

11.3 What is the Euclidean distance between $u = [3, 4]$ and $v = [0, 5]$?

11.4 What is the cosine similarity between $u = [3, 4]$ and $v = [0, 5]$, rounded to 4 decimals?

11.5 Given a batch of embeddings $\mathbf{E}$ of shape (N, D) , what is the shape of the pairwise squared-distance matrix, and name the standard tensor formula $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$ used to compute it efficiently without an $O(N^2)$ Python loop.

11.6 What is `torch.norm(u, p=float('inf'))` (the max-norm) for $u = [3, 4]$?

11.7 For a row-normalization operation `E / E.norm(dim=1, keepdim=True)` on E of shape (N, D) , what is the resulting shape and what property does each row satisfy afterward?

11.8 What is the Frobenius norm of the matrix $\begin{bmatrix} 3 & 4 \\ 0 & 0 \end{bmatrix}$?

11.9 Why is cosine similarity generally preferred over raw dot product for comparing embedding vectors of different magnitudes?

Chapter 12

Autograd & Gradients

- 12.1** Given $y = x^2$ and $x = 3$, what is $\frac{dy}{dx}$ at $x = 3$ (i.e. what does `x.grad` hold after `y.backward()`)?
- 12.2** Given $z = 3x^2 + 2x$ at $x = 1$, what is dz/dx ?
- 12.3** Given $y = \sum_i x_i^2$ for $x = [1, 2, 3]$, what is $\nabla_x y$?
- 12.4** Why does calling `.backward()` twice on the same graph without `retain_graph=True` raise an error?
- 12.5** What does `x.requires_grad_(True)` do to an existing tensor `x`?
- 12.6** Inside a `torch.no_grad()` block, what happens to operations performed on tensors that normally require gradients?
- 12.7** Given $f(x, y) = x^2y + y^3$, what are $\partial f/\partial x$ and $\partial f/\partial y$ at $x = 2, y = 1$?
- 12.8** Why must gradients be zeroed with `optimizer.zero_grad()` before each `backward()` call during training?
- 12.9** Given a leaf tensor `x` with `requires_grad=True` that is

detached via `y = x.detach()`, does `y` share memory with `x`, and does it track gradients?

- 12.10** For a vector-valued function producing $y \in \mathbb{R}^m$ from $x \in \mathbb{R}^n$, what is the shape of the full Jacobian $\partial y / \partial x$, and why does `.backward()` typically require a scalar output instead of computing this full Jacobian?
- 12.11** What is the gradient of $y = \text{ReLU}(x)$ with respect to x at $x = -2$ and at $x = 3$?
- 12.12** Explain in one or two sentences why exploding or vanishing gradients occur in deep networks, in terms of repeated tensor multiplication during backpropagation.

Chapter 13

Advanced & Applied Challenges

- 13.1** Given a batch of images $\mathbf{x}$ of shape (N, C, H, W) , write the einsum/matmul-based expression to compute a fully-connected layer's output where the images have first been flattened, with weight $\mathbf{W}$ of shape $(C \cdot H \cdot W, K)$ and bias $\mathbf{b}$ of shape $(K,)$.
- 13.2** Explain how a 1×1 convolution over a tensor of shape (N, C_{in}, H, W) can be implemented purely as a matrix multiplication after reshaping, without using a convolution operator.
- 13.3** Given multi-head attention with $\mathbf{Q}$ of shape (B, N, D) split into h heads, what reshape/permute sequence turns $\mathbf{Q}$ into shape $(B, h, N, D/h)$?
- 13.4** Given raw attention scores of shape (B, H, N, N) , describe the exact tensor operation sequence to apply a causal (look-ahead) mask before softmax.
- 13.5** Why is dividing attention scores by $\sqrt{D}$ (the head dimension) necessary before the softmax step in scaled dot-product attention?

-
- 13.6** Given a sequence-batch tensor $\mathbf{x}$ of shape (B, N, D) and a padding mask of shape (B, N) marking valid (1) vs padded (0) tokens, write the tensor expression to compute the mean embedding over only the valid tokens per sequence (masked mean pooling), shape (B, D) .
- 13.7** Given a weight tensor of shape (C_{out}, C_{in}, K, K) for a standard 2D convolution, how many total learnable parameters does the convolution layer have, excluding bias, in terms of these dimensions?
- 13.8** Describe, at the tensor-shape level, how the *im2col* technique converts a convolution into a single large matrix multiplication. What are the shapes of the unfolded input patches matrix and the reshaped weight matrix?
- 13.9** Given logits of shape (N, C) and integer labels of shape $(N,)$, write out, step by step in tensor operations, how cross-entropy loss is computed from scratch (softmax + negative log likelihood), without using a built-in loss function.
- 13.10** Given a batch of token embeddings $\mathbf{x}$ of shape (B, N, D) and you want to compute pairwise dot-product similarity between every pair of tokens within each sequence independently (not across the batch), which einsum expression computes this, and what is the output shape?
- 13.11** You are implementing gradient checkpointing to reduce memory during training of a very deep network. At a conceptual tensor/computation-graph level, what is being traded for what?

Answer Key

Tensor Creation & Attributes

1.1 shape (10,), 1 dimension: [0,1,2,3,4,5,6,7,8,9]

1.2 24 elements, rank 3

1.3 dtype float32, shape (5,5)

1.4 shape (4,4); 1s on the main diagonal, 0s elsewhere

1.5 shape (6)

value:

[2, 5, 8, 11, 14, 17]

1.6 shape (5)

value:

[0.0000, 0.2500, 0.5000, 0.7500, 1.0000]

1.7 (3,) is a flat vector of 3 elements; (3,1) is a column matrix with 3 rows and 1 column

1.8 24

1.9 It is truncated toward zero, becoming 3

1.10 shape (2, 3)

value:

[[7, 7, 7], [7, 7, 7]]

1.11 `torch.tensor` always copies the data; `torch.as_tensor` avoids a copy when possible and shares memory with the NumPy array

1.12 `device='cpu', requires_grad=False`

1.13 `torch.randn(3,3)`

1.14 It is a 1-D tensor with one element (not a true scalar); `.item()` extracts that single Python number

1.15 `x.dtype = int64, y.dtype = float32`

Indexing & Slicing

2.1 `shape (6)`

value:

```
[1, 2, 3, 4, 5, 6]
```

2.2 `shape (4)`

value:

```
[ 1, 7, 13, 19]
```

2.3 `shape (2, 3)`

value:

```
[[ 9, 10, 11], [15, 16, 17]]
```

2.4 `24`

2.5 `shape (2, 3)`

value:

```
[[ 1, 3, 5], [13, 15, 17]]
```


2.6 shape (4, 6)

value:

```
[[ 6, 5, 4, 3, 2, 1], [12, 11, 10, 9, 8, 7], [18, 17, 16, 15, 14, 13], [24, 23, 22, 21, 20, 19]]
```

2.7 A flat row vector of shape (6,); basic single-index slicing drops the indexed dimension

2.8 (1, 4, 6) – None inserts a new axis

2.9 A flat 1-D tensor containing only the elements of A greater than 12, in row-major order

2.10 shape (2)

value:

```
[ 2, 16]
```

2.11 shape (3, 6)

value:

```
[[ 1, 2, 3, 4, 5, 6], [ 1, 2, 3, 4, 5, 6], [19, 20, 21, 22, 23, 24]]
```

2.12 Basic slicing (start:stop:step) is expressible as a fixed stride pattern over the original storage, so PyTorch returns a view; boolean/fancy indexing selects arbitrary, non-uniformly-strided elements, which cannot be represented as a view, forcing a new contiguous copy

2.13 Ellipsis fills in all preceding dimensions unchanged; output is A[:,0], shape (4,)

2.14 Row 0 becomes [0,0,0,0,0,0]; since this is a view-based slice assignment, A is modified in place

Reshaping, Views & Permutations

3.1 (4,6)

3.2 (24,) – flattens completely

3.3 (2,12) – the -1 dimension is inferred as 12

3.4 (4,2,3)

3.5 (3,2,4)

3.6 (3,5)

3.7 (3,1,5)

3.8 `.view()` requires the tensor's underlying memory to be contiguous and compatible with the new shape via a simple stride reinterpretation; if the tensor is non-contiguous (e.g. after a transpose), `.view()` fails, whereas `.reshape()` automatically falls back to copying the data when a view is not possible

3.9 No, permute only changes strides/metadata, not memory layout, so `y` is non-contiguous; call `y.contiguous()` first

3.10 (2,12) – flattens dims 1 and 2 together, keeping dim 0

3.11 (2,5,4,3)

3.12 (3,4,2)

3.13 Yes: $6*4=24$ and $2*3*4=24$, element counts match, so the reshape is valid

3.14 `repeat(2,1,1)` gives shape (4,3,4) and physically copies data; `expand` only works on size-1 dimensions, returns a view

with stride 0 along the expanded dimension (no memory copy), so writing into an expanded tensor affects all repeated positions

Broadcasting

4.1 Yes; (4,) is treated as (1,4), aligned and expanded to (3,4); result shape (3,4)

4.2 Yes; both size-1 dimensions expand against each other; result shape (3,4)

4.3 No; aligning from the right, (3,) aligns with the last dim of size 4, and $3 \neq 4$ and neither is 1, so it fails

4.4 (5,4,3)

4.5 Align from the right: (7,1,5) becomes (1,7,1,5); combine with (8,1,6,1) $\rightarrow$ (8,7,6,5)

4.6 Adds the bias vector b to every row of x (broadcast over the batch dimension N); this is exactly the bias-add step of a linear/dense layer

4.7 (4,) aligns with the trailing dimension 4 and broadcasts fine; (3,) also aligns with the trailing dimension of size 4, and since $3 \neq 4$ and neither is 1, broadcasting fails

4.8 (4,3,5)

4.9 mean of shape (3,1,1) aligns with the last three dims (3,H,W) of images, broadcasting across N and across H,W , subtracting the correct channel mean from every pixel

4.10 If you meant a per-sample vector of shape (N,) but it has shape (N,1) instead of your target (N,D), adding it to a (N,D) tensor still broadcasts successfully but adds the same scalar per row across the D axis, silently doing something different than the intended elementwise add, with no shape error raised

Elementwise & Arithmetic Operations

5.1 shape (4)

value:

```
[1.0000, 2.0000, 3.0000, 4.0000]
```

5.2 shape (4)

value:

```
[ 1.0000, 4.0000, 9.0000, 16.0000]
```

5.3 shape (4)

value:

```
[1.0000, 0.0000, 3.0000, 0.0000]
```

5.4 shape (4)

value:

```
[ 1.0000, -1.0000, 1.0000, -1.0000]
```

5.5 shape (4)

value:

```
[ 2.0000, -4.0000, 6.0000, -8.0000]
```

5.6 shape (4)

value:

```
[ 0.5000, -1.0000, 1.5000, -2.0000]
```

5.7 shape (4)

value:

```
[ 1.0000, -1.0000, 2.0000, -1.0000]
```

5.8 shape (4)

value:

```
[0.7311, 0.1192, 0.9526, 0.0180]
```

5.9 shape (4)

value:

```
[ 0.7616, -0.9640, 0.9951, -0.9993]
```

5.10 shape (4)

value:

```
[ 2.7183, 0.1353, 20.0855, 0.0183]
```

5.11 `x.add(1)` returns a new tensor leaving `x` unchanged; `x.add_(1)` is an in-place operation (trailing underscore convention) that mutates `x` directly and returns the same modified tensor

5.12 shape (4)

value:

```
[ 1.0000, -0.0200, 3.0000, -0.0400]
```

5.13 shape (4)

value:

```
[1, 2, 0, 1]
```

5.14 shape (4)

value:

```
[ 1.0000, -8.0000, 27.0000, -64.0000]
```

5.15 PyTorch's `/` operator always performs true division and promotes to float, giving `tensor([0.5, 1.0, 1.5])` with dtype float32,

even though inputs were integer tensors

Reduction Operations

6.1 21.0

6.2 shape (3)

value:

[5.0000, 7.0000, 9.0000]

6.3 shape (2)

value:

[6.0000, 15.0000]

6.4 (2,1)

6.5 3.5

6.6 values: [3., 6.], indices: [2, 2]

6.7 5

6.8 shape (3)

value:

[2.2500, 2.2500, 2.2500]

6.9 shape (3)

value:

[1.5000, 1.5000, 1.5000]

6.10 shape (2)

value:

[6.0000, 120.0000]

6.11 `shape (2, 3)`

value:

```
[[ 1.0000,  3.0000,  6.0000], [ 4.0000,  9.0000, 15.0000]]
```

6.12 Without `keepdim` the reduced dimension is removed entirely (`shape (2,)`); with `keepdim=True` the reduced dimension is kept but set to size 1 (`shape (2,1)`), which is useful for subsequent broadcasting

6.13 `logsumexp(M,dim=1)` computes `log(sum(exp(row)))` for each row in a numerically stable way by subtracting the row max before exponentiating and adding it back after taking the log, avoiding overflow that the naive `exp-then-log-then-sum` formula suffers for large values

6.14 `all(M>0) = True` (every entry is positive); `any(M>5) = True` (6 is greater than 5)

Matrix Multiplication & Linear Algebra

7.1 `shape (2, 2)`

value:

```
[[19.0000, 22.0000], [43.0000, 50.0000]]
```

7.2 `shape (2, 2)`

value:

```
[[ 5.0000, 12.0000], [21.0000, 32.0000]]
```

7.3 `shape (2, 2)`

value:

```
[[1.0000, 3.0000], [2.0000, 4.0000]]
```

7.4 `shape (2)`

value:

[3.0000, 7.0000]

7.5 -2.0000000000000004

7.6 shape (2, 2)

value:

[[-2.0000, 1.0000], [1.5000, -0.5000]]

7.7 5.0

7.8 (16, 4, 3) – batched matrix multiplication multiplies each of the 16 pairs independently

7.9 (4,3)

7.10 Shape (2,4,3); torch.matmul broadcasts the batch dimensions, treating Y as if it were repeated for every batch element in X, since matmul applies standard broadcasting rules to all dimensions except the last two

7.11 At most 2 – the matrix is rank-deficient because two rows are linearly dependent

7.12 shape (2)

value:

[-0.3723, 5.3723]

7.13 5.477225575051661

7.14 torch.mm is a strict 2-D matrix multiply with no broadcasting support, intended for simple, unambiguous matrix products; torch.matmul is the general-purpose operator that additionally supports batched matmul (broadcasting all leading

batch dimensions) and even matrix-vector / vector-vector products, so it subsumes mm's functionality for higher-rank inputs

Concatenation, Stacking & Splitting

8.1 (4,2)

8.2 (2,4)

8.3 Shape (2,2,2); stack introduces a brand-new dimension joining whole tensors together, while cat joins tensors along an already-existing dimension without adding a new one

8.4 shape (4, 2)

value:

```
[[1, 2], [3, 4], [5, 6], [7, 8]]
```

8.5 shape (2, 2, 2)

value:

```
[[[3, 4], [7, 8]]][[[1, 2], [5, 6]],  
[[3, 4], [7, 8]]]
```

8.6 Three tensors each of shape (2,4)

8.7 Four chunks: sizes (2,4), (2,4), (1,4), (1,4) -- chunk divides as evenly as possible, with earlier chunks getting the extra elements

8.8 Concatenation only extends the chosen axis; every other axis must line up so that the resulting rows/columns/slices remain aligned and rectangular -- any mismatch in a non-concate

dimension would make the result ill-defined (ragged), so PyTorch raises an error

8.9 `torch.split(x, 5, dim=0)`; yields two tensors each of shape (5,3,32,32)

Advanced Indexing: Gather, Scatter, Masking

9.1 `shape (3, 1)`
`value:`
`[[10], [60], [80]]`

9.2 It builds an output tensor the same shape as `index`, where each output element is picked from input along the given `dim` using the corresponding entry of `index` as the coordinate on that `dim`, keeping all other coordinates the same

9.3 `scores.gather(1, labels.unsqueeze(1)).squeeze(1)`
-- `unsqueeze labels` to shape (N,1) to match `gather`'s index-shape requirement, then `squeeze` the result back to shape (N,)

9.4 `scatter_` is the inverse operation: instead of reading values out along an index, it writes values from `src` into the calling tensor at positions specified by `index` along the given `dim`, in place

9.5 `onehot = torch.zeros(N, C)`; `onehot.scatter_(1, labels.unsqueeze(1))` -- writes a 1 into each row at the column given by that row's label

9.6 `shape (3, 3)`

value:

```
[[ 0, 0, 0], [ 0, 0, 60], [70, 80, 90]]
```

9.7 `masked_select` returns a flattened 1-D tensor containing only the True-masked elements (variable length, shape changes); where preserves the original shape entirely, choosing element-wise between M and other depending on the mask

9.8 `shape (3, 3)`

value:

```
[[70, 80, 90], [10, 20, 30], [40, 50, 60]]
```

9.9 `gather` is a single vectorized GPU-parallel operation that avoids Python-level looping over the batch dimension, which would be far slower and would break the computation graph needed for autograd if done via plain indexing in a loop

Einsum & Tensor Contractions

10.1 Standard matrix multiplication `A @ B`

10.2 `shape (2, 2)`

value:

```
[[19.0000, 22.0000], [43.0000, 50.0000]]
```

10.3 The trace of A (sum of diagonal elements)

10.4 5.0

10.5 The transpose of A

10.6 11.0

10.7 The outer product of v and w , producing a matrix of shape $(\text{len}(v), \text{len}(w))$

10.8 `shape (2, 2)`

value:

```
[[3.0000, 4.0000], [6.0000, 8.0000]]
```

10.9 `torch.einsum('bhqd,bhkd->bhqk', Q, K) -- contracts over the head-dimension  $d$  while keeping batch  $b$  and heads  $h$  as independent batch axes, producing an  $(N,N)$  score matrix per batch/head`

10.10 `torch.einsum('bhqk,bhkd->bhqd', attn, V)`

10.11 Batched matrix multiplication -- equivalent to `torch.bmm(X, Y)`, producing shape $(B,4,3)$

10.12 `einsum` lets you specify exactly which axes are summed (contracted), broadcast, or kept in a single declarative expression, which is often clearer, less error-prone, and can be more efficiently fused by the backend than an equivalent sequence of separate `reshape`/`permute`/`matmul` calls

Norms, Distances & Similarity

11.1 5.0

11.2 7.0

11.3 3.1622776601683795

11.4 0.8

11.5 `Shape (N,N); computed as sq_norms[:,None] + sq_norms[None,:]`
`- 2 * E @ E.T, where sq_norms = (E**2).sum(dim=1)`

11.6 4.0

11.7 Shape stays (N,D); each row now has unit L2 norm (length 1), used e.g. before computing cosine similarities via a plain dot product

11.8 5.0

11.9 Raw dot product is scale-sensitive -- a vector with a larger magnitude can produce a bigger dot product purely because of its length, not because it's more semantically similar; cosine similarity normalizes by both vectors' norms, isolating the angle between them so comparisons reflect direction/similarity rather than magnitude

Autograd & Gradients

12.1 $dy/dx = 2x = 6$

12.2 $dz/dx = 6x+2 = 8$ at $x=1$

12.3 [2,4,6]

12.4 By default PyTorch frees the intermediate buffers used to compute gradients as soon as `backward()` is called, to save memory; a second `backward()` call has nothing left to compute gradients from unless `retain_graph=True` was passed to preserve those buffers

12.5 It marks `x` (in place) so that operations on it are tracked by autograd, enabling gradient computation with respect to `x`

12.6 Autograd tracking is disabled -- the resulting tensors have `requires_grad=False` and no computation graph is built, which saves memory and speeds up inference/evaluation code

12.7 $df/dx = 2xy = 4$; $df/dy = x^2 + 3y^2 = 4 + 3 = 7$

12.8 PyTorch accumulates gradients into `.grad` by default (adds to existing values rather than overwriting) across multiple `backward()` calls, so without zeroing, gradients from previous batches would incorrectly add onto the current batch's gradients

12.9 Yes, `y` shares the same underlying storage as `x` (same data, no copy), but `y` is detached from the autograd graph, so `y.requires_grad` is `False` and gradients will not flow through `y`

12.10 The Jacobian has shape `(m,n)`; `backward()` implements reverse-mode differentiation which efficiently computes vector-Jacobian products for a scalar (or given upstream gradient vector) rather than materializing the full `(m,n)` Jacobian, which would be far more expensive for large `m` and `n`

12.11 At `x=-2`, gradient is 0 (function is flat/zero there); at `x=3`, gradient is 1 (identity slope in the positive region)

12.12 Backpropagation through many layers multiplies

together many Jacobian/weight-derivative matrices; if their dominant singular values are consistently greater than 1 the product grows exponentially with depth (exploding), and if consistently less than 1 it shrinks exponentially toward zero (vanishing)

Advanced & Applied Challenges

13.1 `out = x.reshape(N, C*H*W) @ W + b`, giving shape (N, K) ; the `flatten` collapses all non-batch dims, the `matmul` projects to K output features, and the bias broadcasts across the batch

13.2 Reshape/permute x to shape $(N*H*W, C_{in})$, multiply by a weight matrix of shape (C_{in}, C_{out}) , giving $(N*H*W, C_{out})$, then reshape/permute back to (N, C_{out}, H, W) ; a 1×1 convolution has no spatial receptive field beyond a single pixel, so it reduces exactly to an independent linear layer applied at every spatial location

13.3 `Q.reshape(B, N, h, D//h).permute(0, 2, 1, 3)` -- first split the feature dim D into $(h, D/h)$, then move the head dimension next to the batch dimension so each head can be treated as an independent batch element

13.4 Build a boolean upper-triangular mask of shape (N, N) marking future positions, broadcast it to (B, H, N, N) (or rely on broadcasting), and use `scores.masked_fill(mask, float('-inf'))` to set disallowed positions to $-\infty$ so that after softmax they receive weight 0

13.5 As D grows, the dot products $Q \cdot K^T$ tend to have large variance (one-hot outputs) and vanishing gradients elsewhere; dividing by $\sqrt{D}$ keeps

13.6 `mask = pad_mask.unsqueeze(-1).float(); pooled = (x * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1e-9),` giving shape (B,D) -- multiply out padded positions before summing, then divide by the count of valid tokens per sequence rather than N

13.7 `C_out * C_in * K * K`

13.8 Each $K \times K \times C_{in}$ receptive-field patch at every output location is unfolded into a column, producing a matrix of shape $(C_{in} \times K \times K, N_{out_positions})$ per image (or $(N, C_{in} \times K \times K, L)$ batched); the weight tensor is reshaped from (C_{out}, C_{in}, K, K) to $(C_{out}, C_{in} \times K \times K)$; multiplying `weight_matrix @ patches_matrix` gives shape $(C_{out}, N_{out_positions})$ which is reshaped back into the output feature map $(C_{out}, H_{out}, W_{out})$

13.9 `probs = softmax(logits, dim=1); correct_probs = probs.gather(1, labels.unsqueeze(1)).squeeze(1); loss = -torch.log(correct_probs).mean()` -- normalize logits into a probability distribution per row, pick out the probability assigned to the true class via gather, take the negative log (higher confidence in the correct class gives lower loss), and average over the batch

13.10 `torch.einsum('bnd,bmd->bnm', x, x);` output shape (B,N,N) -- batch dimension b is kept independent, feature dimension d is contracted (summed over), producing an $N \times N$ similarity matrix per sequence in the batch

13.11 Memory is traded for compute: instead of storing every intermediate activation tensor needed for the backward pass, only a subset of checkpoints are kept, and the discarded

intermediate activations are recomputed via a second forward pass through that segment during backpropagation, reducing peak memory usage at the cost of extra forward computation